Sprawozdanie – Laboratorium 3: PostgreSQL, migracje, relacje i transakcje

Przedmiot: Zaawansowane Technologie Programowania Aplikacji Internetowych

Temat: Room Booking API (Relacyjna baza danych i współbieżność)

Student: Kacper Gumulak – 151872

GitHub: github.com/zephir-x/pk-ztpai

1. Krótki opis rozwiązania

Zadanie polegało na ewolucji architektury aplikacji z modelu przechowującego stan w pamięci operacyjnej na model wykorzystujący trwałą, relacyjną bazę danych PostgreSQL uruchomioną w środowisku Docker. Integrację z bazą danych zrealizowano za pomocą Query Buildera Knex.js.

Zaprojektowano strukturę relacyjną składającą się z budynków, sal oraz rezerwacji. Za zarządzanie schematem bazy odpowiadają wersjonowane migracje, a bazę wypełniono przykładowymi danymi za pomocą idempotentnego skryptu (seed). Dzięki elastycznej architekturze wstrzykiwania zależności (Dependency Injection) z poprzedniego laboratorium, warstwę repozytoriów podmieniono bez ingerencji w moduł logiki biznesowej, wdrażając jednocześnie wzorzec Unit of Work. Zoptymalizowano zapytania bazy danych, eliminując problem N+1 poprzez zastosowanie klauzul JOIN oraz przeniesiono mechanizmy paginacji, filtrowania i sortowania bezpośrednio do silnika PostgreSQL. Szczególną uwagę poświęcono zjawisku "Race Condition" (Double Booking), zabezpieczając aplikację na dwóch poziomach: aplikacyjnym (Pessimistic Locking) oraz bazodanowym (Exclusion Constraints).

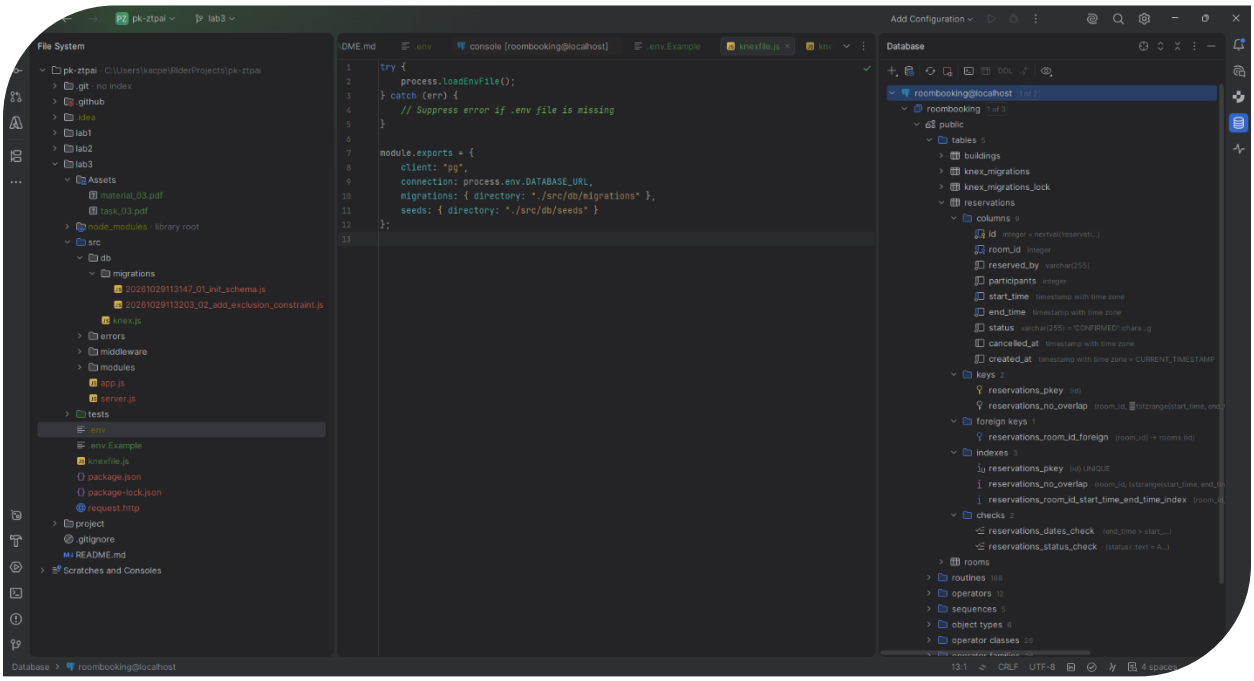
2. Realizacja zadań

Zadanie 1: Baza w Dockerze i migracje

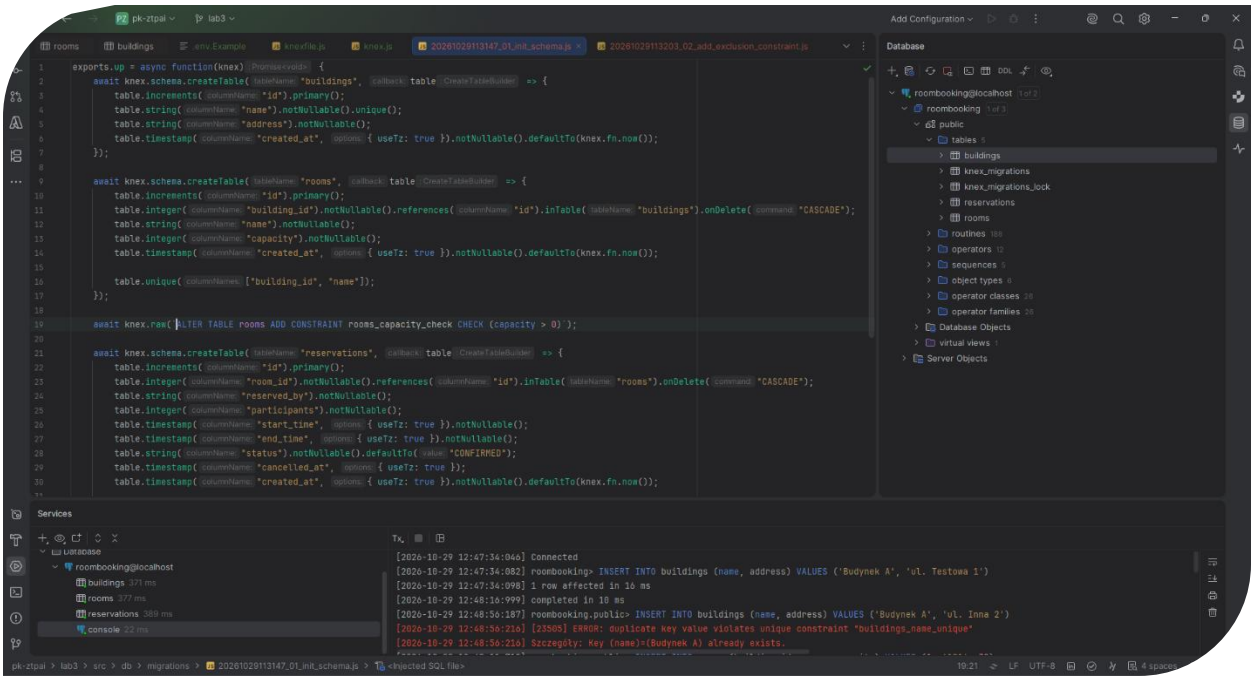
Uruchomiono kontener z oficjalnym obrazem `postgres:16`, podpinając zewnętrzny wolumen w celu zapewnienia persystencji danych. Zdefiniowano migracje tworzące tabele: `buildings`, `rooms` oraz `reservations`. Zastosowano kaskadowe usuwanie powiązanych rekordów (`ON DELETE CASCADE`). Decyzja ta podyktowana jest logiką domeny: sala bez fizycznego budynku nie ma racji bytu, a w przypadku likwidacji sali (np. z powodu remontu), utrzymywanie "osieroconych" rezerwacji na nieistniejący zasób mijałoby się z celem. Spójność danych została zabezpieczona na poziomie silnika bazy za pomocą licznych constraintów (UNIQUE, CHECK).

Fragment kodu (Migracja definiująca relacje, kaskady i Checki):

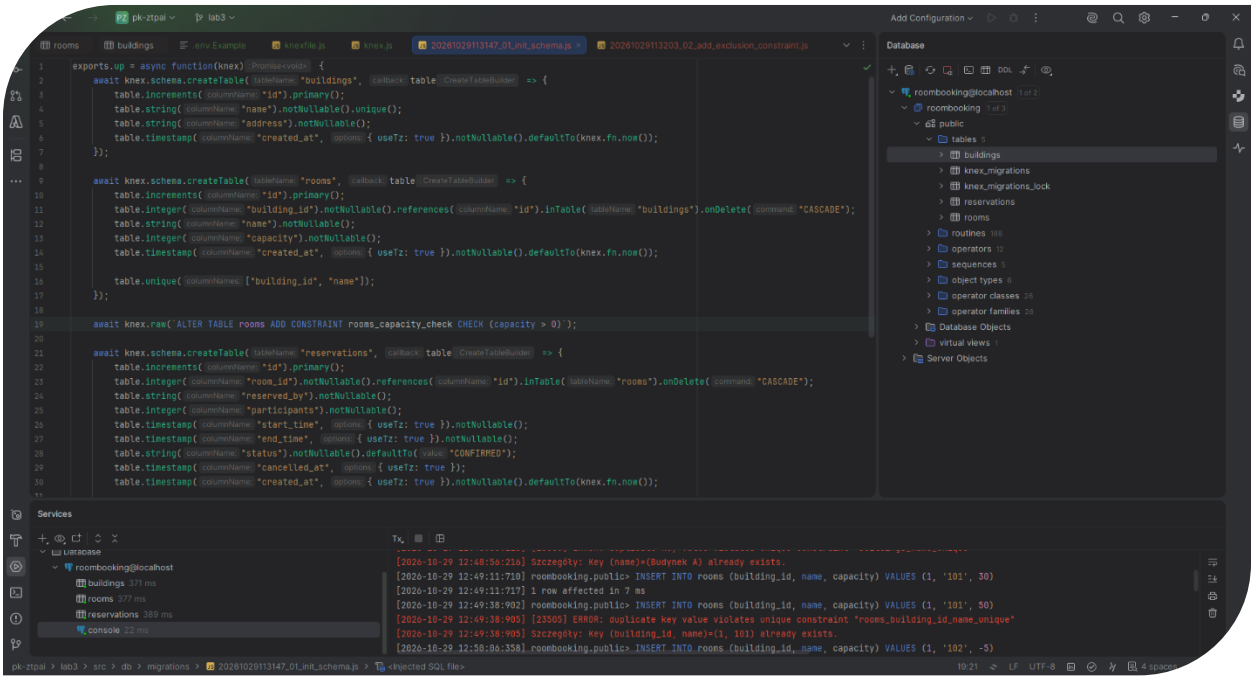
```
exports.up = async function(knex) {
    // ... (tabela buildings)
    await knex.schema.createTable("rooms", table => {
        table.increments("id").primary();
        table.integer("building_id").notNullable().references("id").inTable("buildings").onDelete("C
        ASCADE");
        table.string("name").notNullable(); table.integer("capacity").notNullable();
        table.timestamp("created_at", { useTz: true }).notNullable().defaultTo(knex.fn.now());
        table.unique(["building_id", "name"]);
    });
    await knex.raw(`ALTER TABLE rooms ADD CONSTRAINT rooms_capacity_check CHECK (capacity > 0)`);
    // ... (tabela reservations i indeksy)
    await knex.raw(`ALTER TABLE reservations ADD CONSTRAINT reservations_dates_check CHECK
(end_time > start_time)`);
};
```



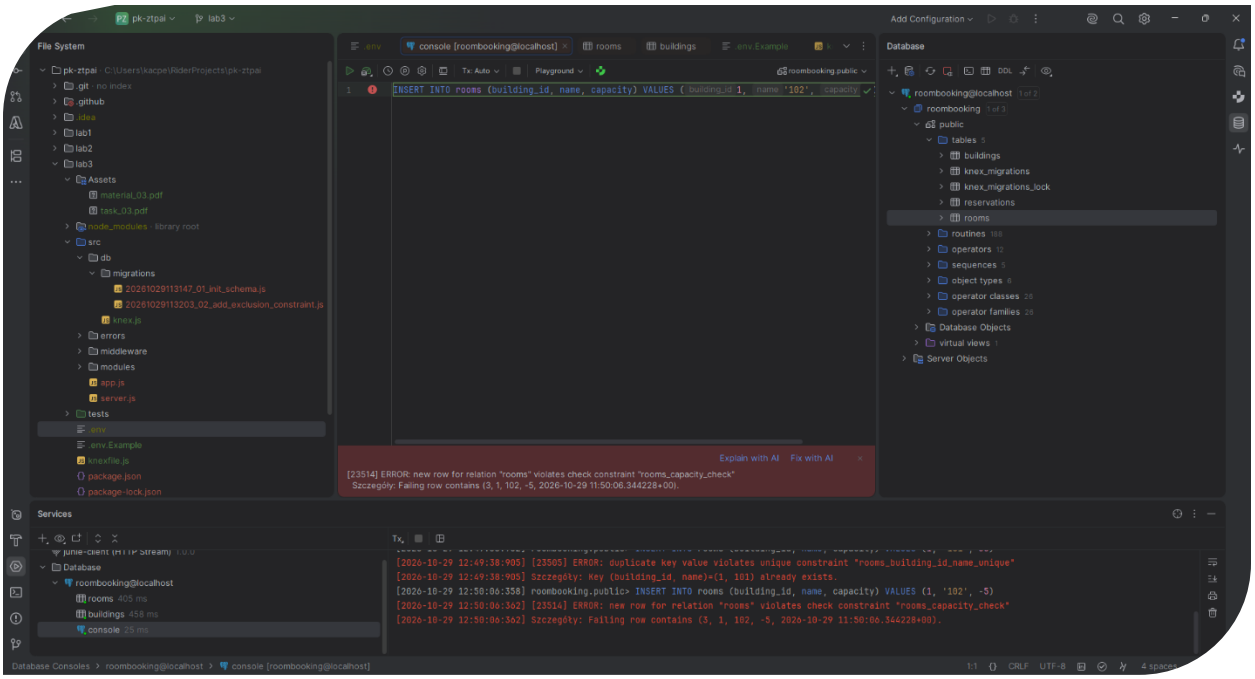
Zrzut ekranu z klienta bazy danych ukazujący strukturę tabeli reservations (\d reservations).



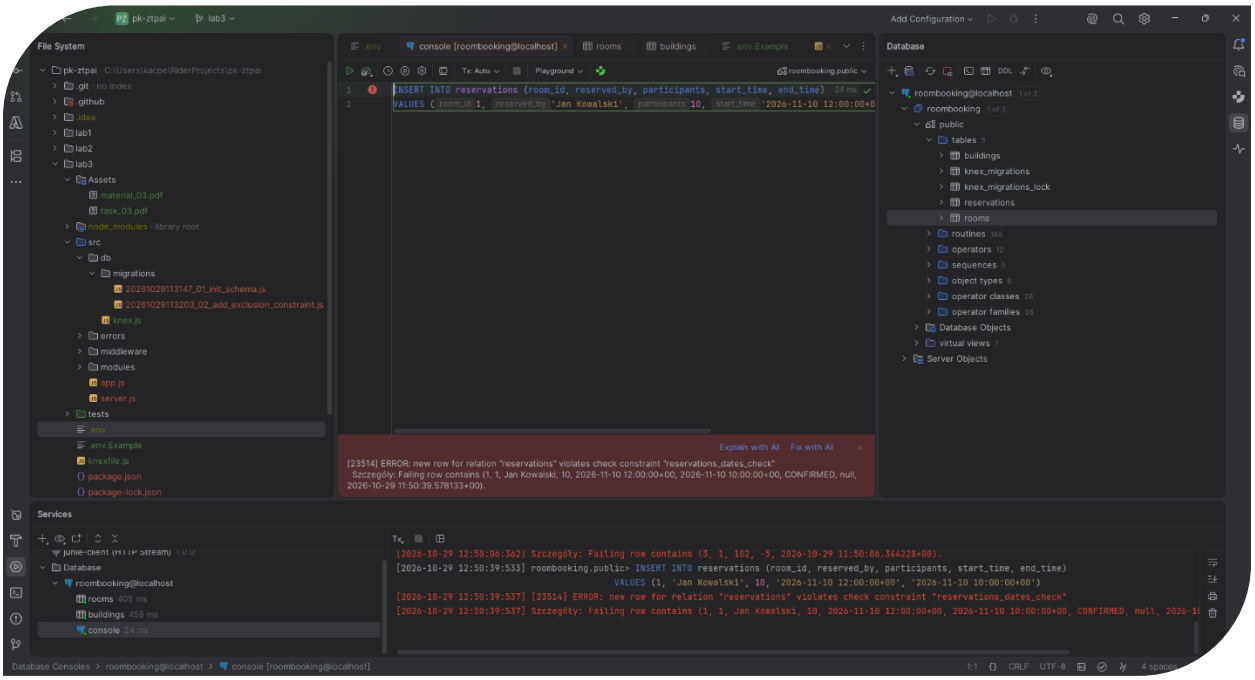
Zrzut ekranu ukazujący próbę złamania constraintu UNIQUE dla nazwy budynku.



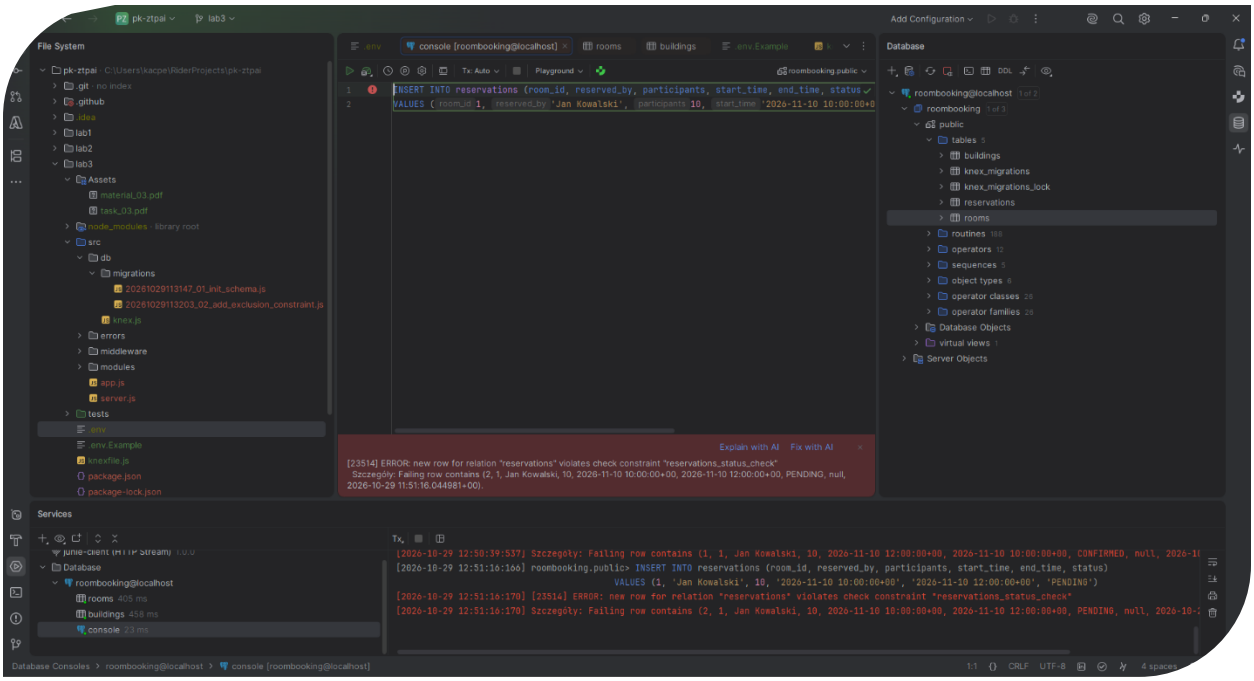
Zrzut ekranu ukazujący próbę utworzenia pokoju o istniejącej już nazwie w danym budynku.



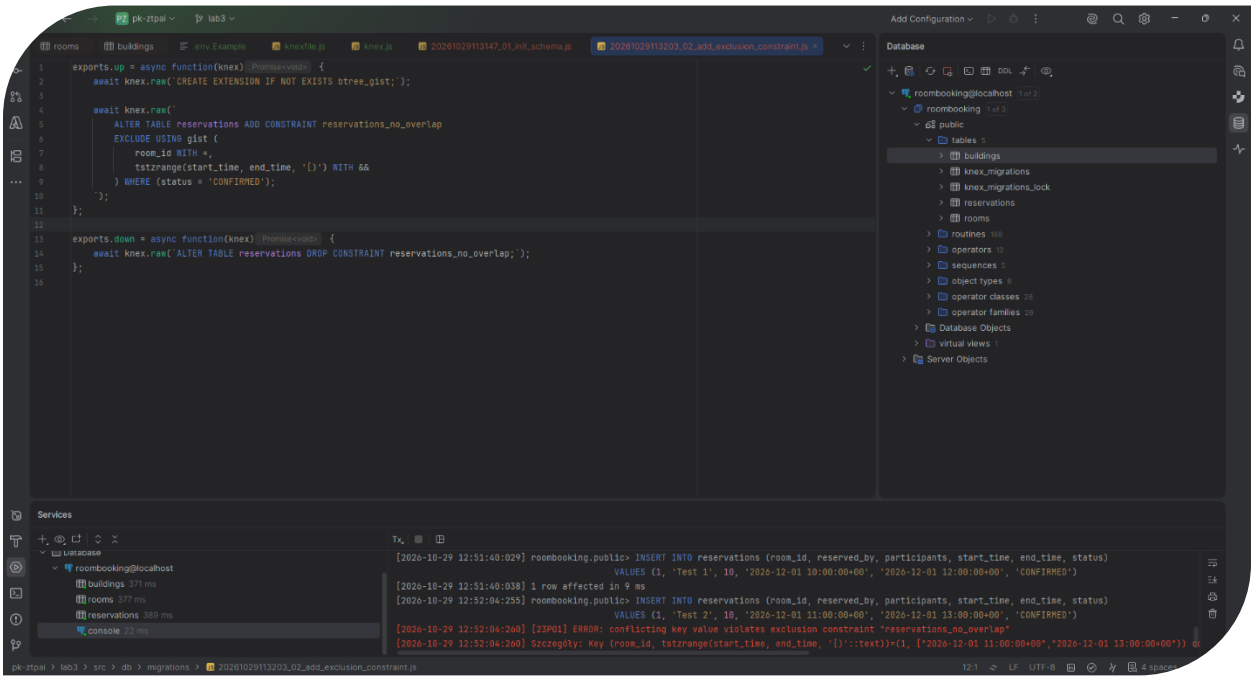
Zrzut ekranu ukazujący próbę wpisania ujemnej pojemności sali (-5), zablokowaną przez CHECK.



Zrzut ekranu ukazujący próbę utworzenia rezerwacji z datą zakończenia mniejszą niż data rozpoczęcia.



Zrzut ekranu ukazujący próbę wpisania niedozwolonego statusu rezerwacji.



Zrzut ekranu udowadniający działanie EXCLUDE USING gist (btree_gist) blokującego nakładające się daty.

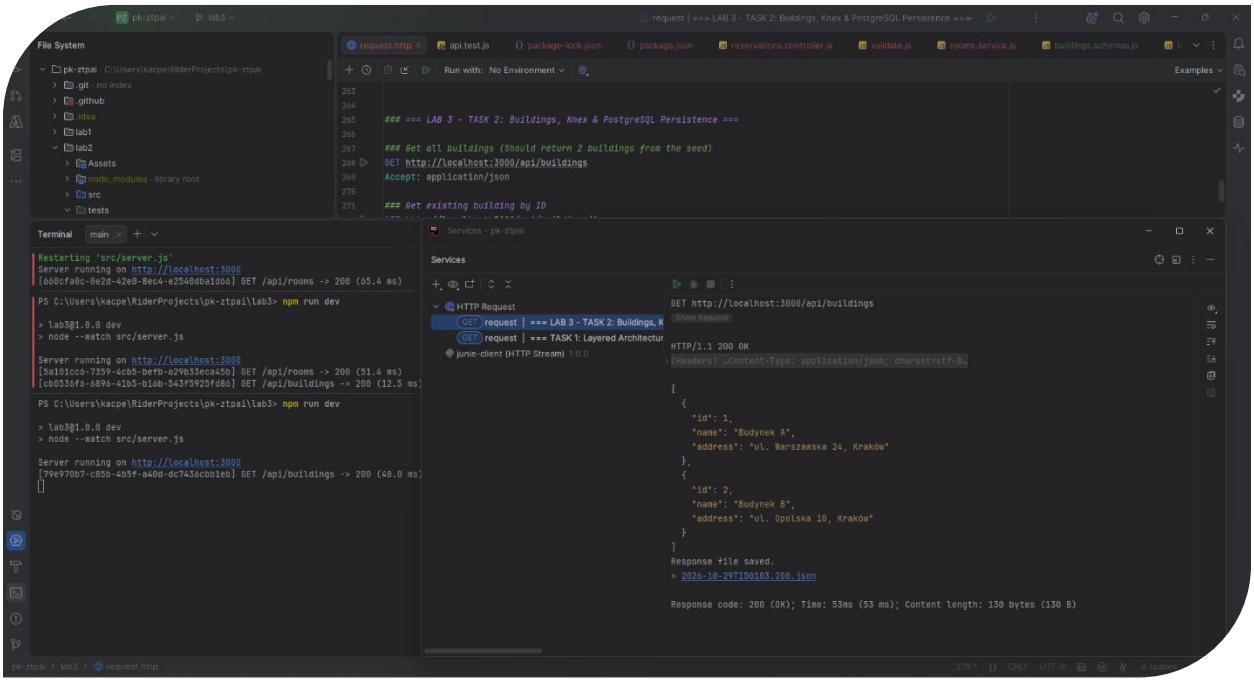
Zadanie 2: Seeds i repositories na Knex

Stworzono skrypt seedujący czyszczący bazę (TRUNCATE ... RESTART IDENTITY CASCADE) i dodający domyślne dane. Utworzono nowe repozytoria wykorzystujące bibliotekę Knex. Repozytoria stały się jedynym miejscem, gdzie obiekty bazodanowe snake_case tłumaczone są na aplikacyjne DTO w konwencji camelCase. Obsłużono wyciek informacji z silnika bazy – kod błędu PostgreSQL 23505 (UNIQUE_VIOLATION) jest przechwytywany w centralnym middleware i formatowany jako kod HTTP 409 Conflict. Zaimplementowano wzorzec Unit of Work, wstrzykując metodę runInTransaction do serwisów.

Fragment kodu (git diff / zmiana w rooms.service.js):

```
-// Wersja In-Memory (Lab 2)
-function createRoomsService({ roomsRepository }) {
-    async function getRoom(id) {
-        const room = await roomsRepository.findById(id);
-        if (!room) throw new NotFoundError("Room");
-        return room;
-    }

+// Wersja transakcyjna Knex z Unit Of Work (Lab 3)
+function createRoomsService({ runInTransaction }) {
+    async function getRoom(id) {
+        return runInTransaction(async ({ rooms }) => {
+            const room = await rooms.findById(id);
+            if (!room) throw new NotFoundError("Room");
+            return room;
+        });
+    }
```



Zrzut ekranu z terminala i zapytania HTTP udowadniający, że dzięki wolumenowi Dockera dane

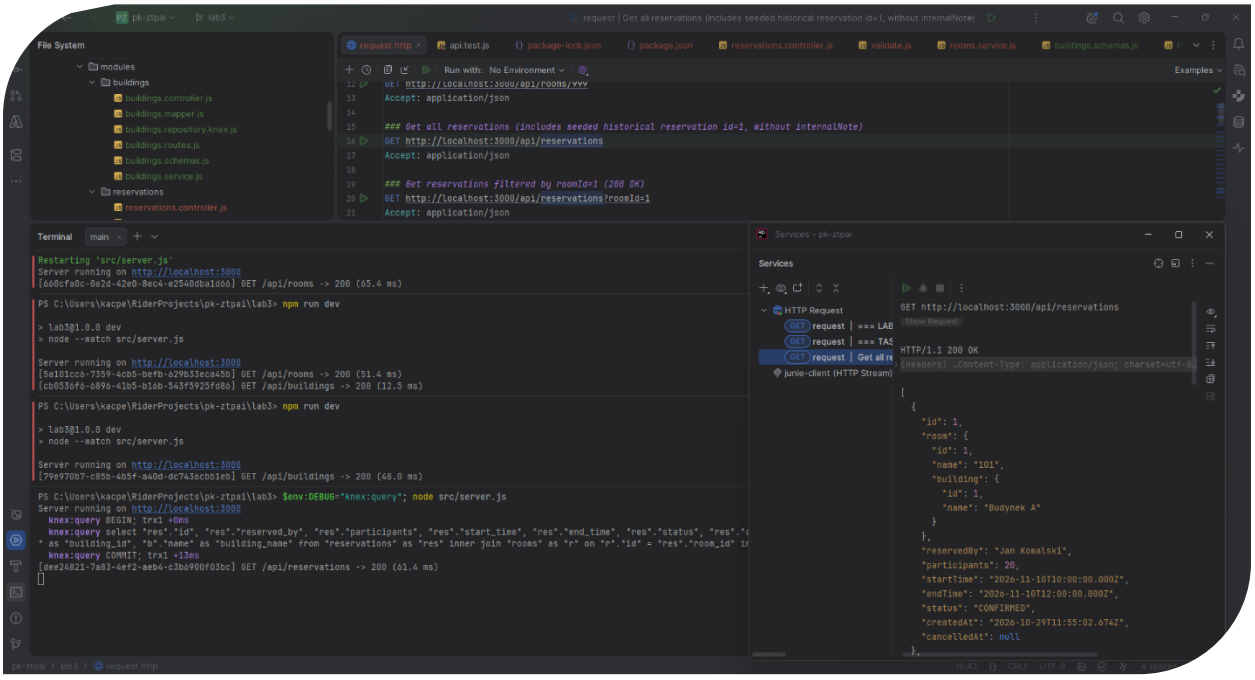
sal i budynków są dostępne po całkowitym restarcie procesu Node.js.

Zadanie 3: Relacje i JOIN bez N+1

Wyeliminowano obciążający bazę danych problem N+1 (wysyłanie osobnych kwerend dla podzasobów w pętli). Przepisano zapytania w repozytoriach tak, aby używały złączeń JOIN. Rozbudowano mappery, które konwertują płaski zbiór kolumn na zagnieżdżoną strukturę JSON. Dla podsumowania pokoi w budynku wykorzystano LEFT JOIN połączony z COUNT i GROUP BY, dzięki czemu budynki bez sal poprawnie zwracają wartość 0, zamiast znikać z wyników.

Fragment kodu (Eliminacja N+1 w repozytorium rezerwacji):

```
const getBaseQuery = () => db("reservations as res")
    .join("rooms as r", "r.id", "res.room_id")
    .join("buildings as b", "b.id", "r.building_id")
    .select(
        "res.id", "res.reserved_by", "res.start_time", "res.status",
        "r.id as room_id", "r.name as room_name",
        "b.id as building_id", "b.name as building_name"
    );
```



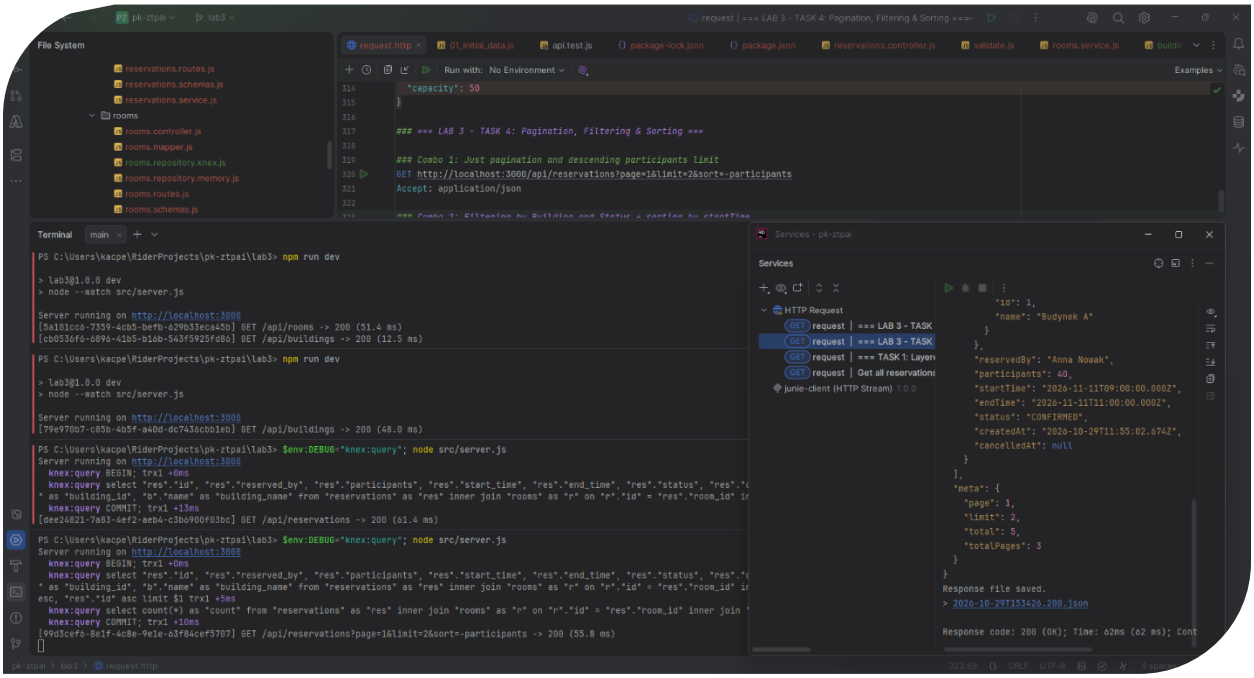
Zrzut ekranu przedstawiający wygenerowane zapytanie w konsoli dla 5 pobranych rezerwacji (1 potężne zapytanie SQL).

Zadanie 4: Pagination, filtrowanie i sortowanie

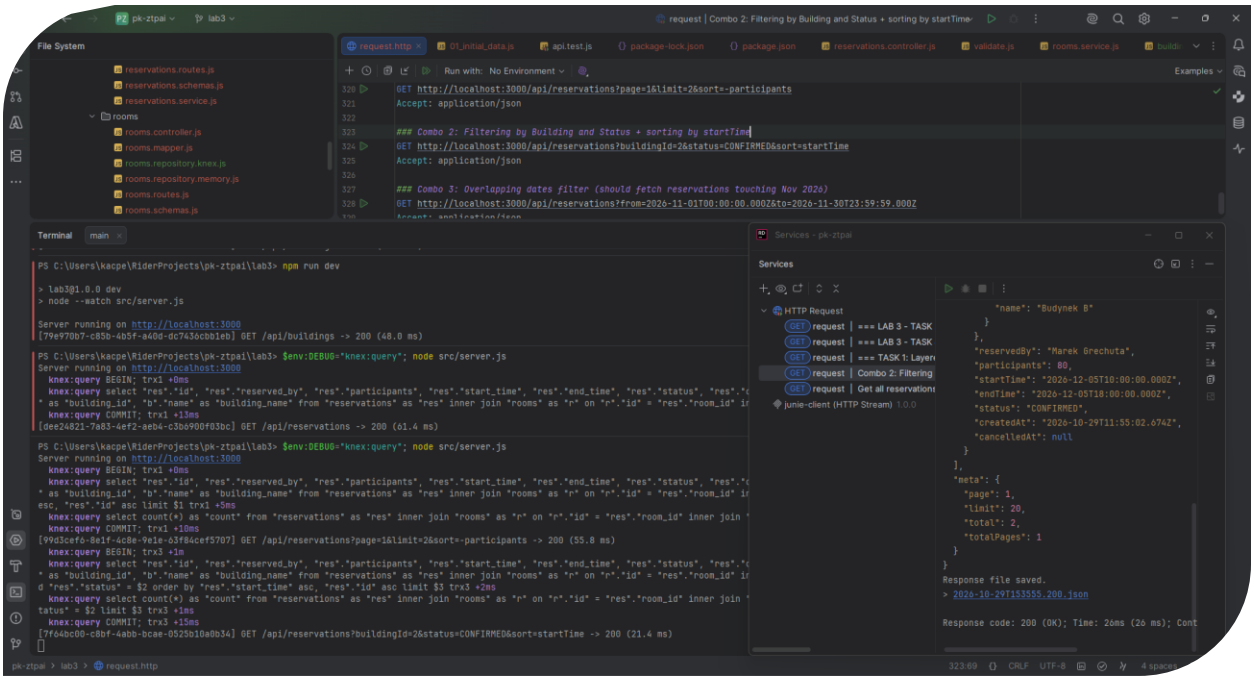
Przeniesiono logikę filtrowania z języka JavaScript bezpośrednio do klauzul bazodanowych. Zabezpieczono sortowanie używając bezpiecznej białej listy w obiekcie mapującym (SORT_COLUMNS), co zapobiega błędom typu SQL Injection. W celu optymalizacji i poprawnego wyliczenia ilości stron (metadane total i totalPages), wykorzystano równoległe wywołanie Promise.all klonujące bazowe zapytanie dla poleceń limit/offset oraz count. W walidacji Zod zaimplementowano regułę weryfikującą kolejność podawanych dat wyszukiwania.

Fragment kodu (Bezpieczna walidacja Zod dla parametrów from i to):

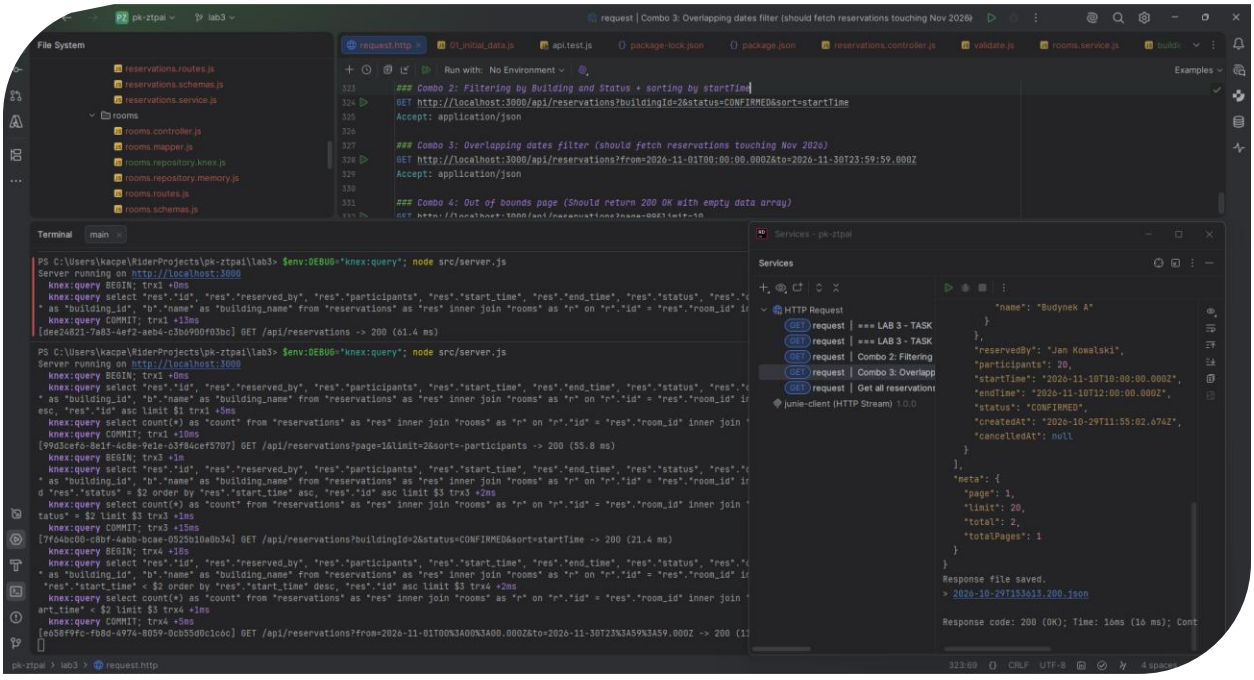
```
const listReservationsQuery = z.object({
    // ... (inne parametry limit, page, sort)
    from: z.iso.datetime({ offset: true }).optional(),
    to: z.iso.datetime({ offset: true }).optional(),
}).refine(data => {
    if (data.from && data.to) return new Date(data.from) < new Date(data.to);
    return true;
}, { message: "from parameter must be earlier than to parameter", path: ["from"] });
```



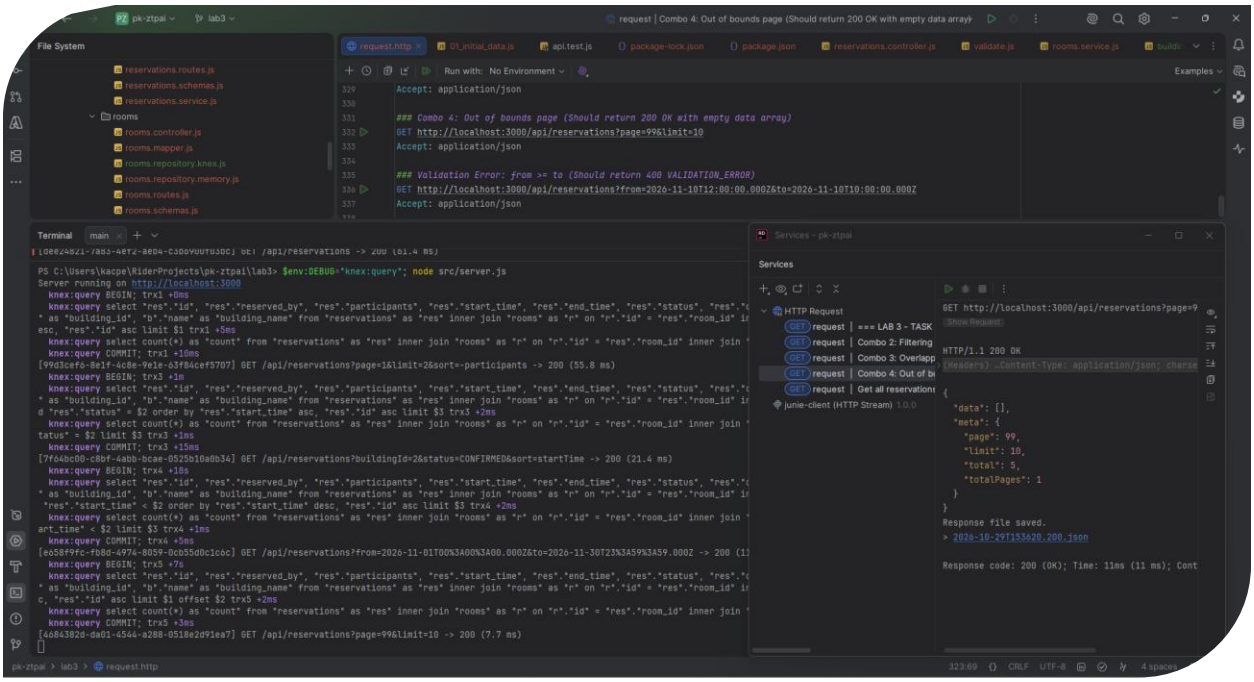
Zrzut ekranu z zapytania Combo 1: paginacja oraz bezpieczne sortowanie malejące po liczbie uczestników.



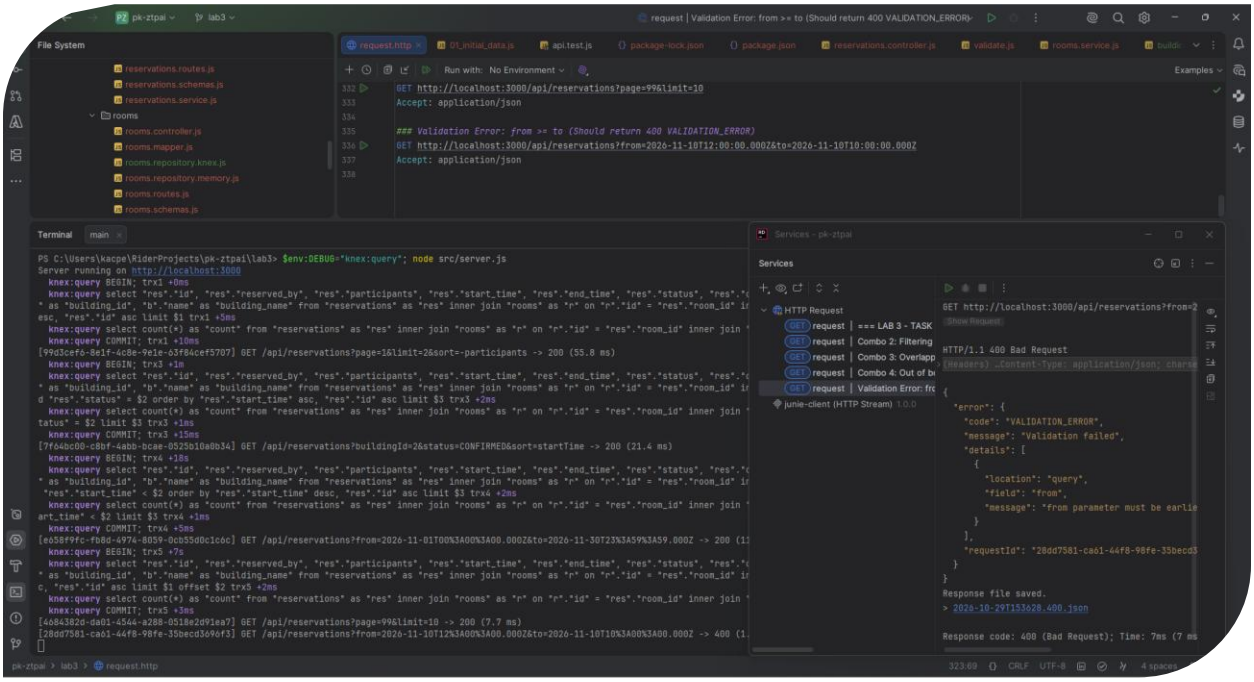
Zrzut ekranu z zapytania Combo 2: filtrowanie rezerwacji z konkretnego budynku i po konkretnym statusie.



Zrzut ekranu z zapytania Combo 3: zaawansowane filtrowanie nakładających się przedziałów (rezerwacje "zahaczające" o miesiąc listopad).



Zrzut ekranu z zapytania Combo 4: poprawna obsługa strony poza zakresem (zwracany kod 200 OK z pustą tablicą).



Zrzut ekranu z zapytania Combo 5: błąd walidacji Zod z powodu podania parametru 'from' późniejszego niż 'to'.

Zadanie 5: Double booking - Race condition

Przeprowadzono analizę bezpieczeństwa aplikacji narażonej na jednoczesne, asynchroniczne żądania rezerwacji tej samej sali (wzorzec *check-then-act*). Wprowadzono skrypt testowy, który udowodnił wygenerowanie duplikatów przy braku blokad. Problem rozwiązano, wdrażając podwójną warstwę ochrony.

W kodzie aplikacji wprowadzono blokadę *Pessimistic Locking* – rezerwowany jest wiersz konkretnej Sali. Wybrano wiersz Sali (a nie samych rezerwacji), ponieważ reguła spójności harmonogramu tyczy się całego zasobu pokoju – żaden inny proces nie może modyfikować jej terminarza, póki obecny wątek nie zweryfikuje nakładania się dat i nie sfinalizuje zapisu. Jako ostateczną barierę zastosowano również barierę bazodanową (btree_gist z operatorem &&).

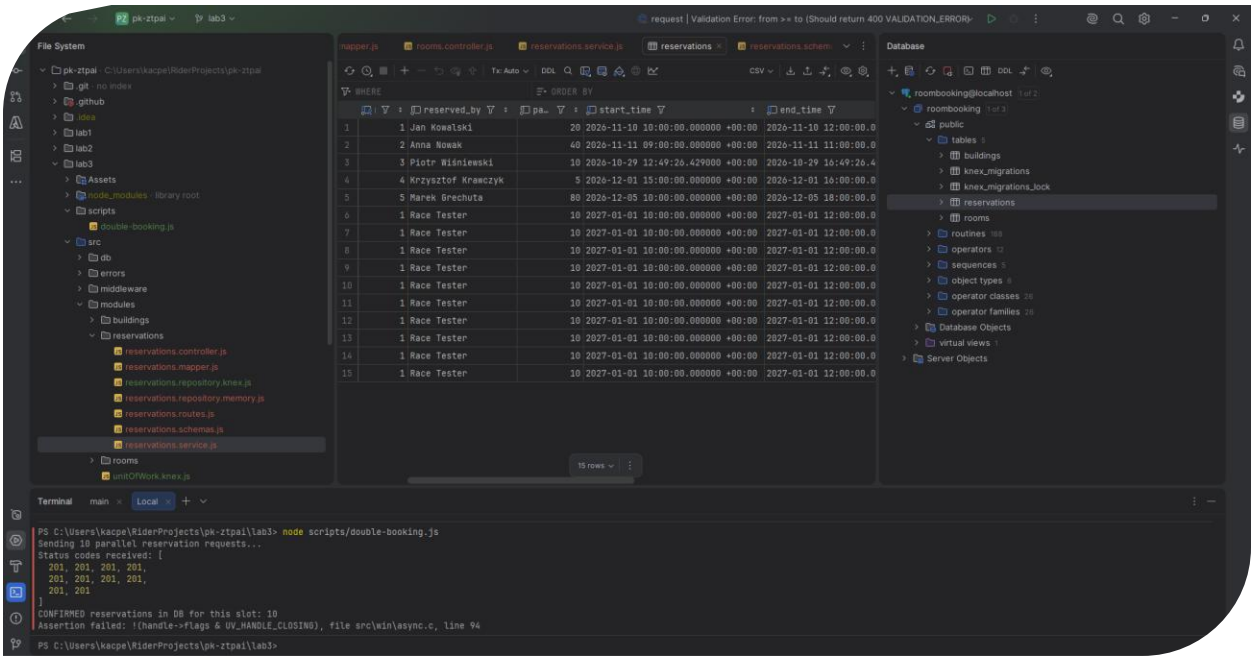
Uzasadnienie zjawiska przy braku blokad: Domyślny poziom izolacji w bazie PostgreSQL, z którego korzysta Knex.js, to **Read Committed**. Zapewnia on atomowość i rozwiązuje problem "brudnych odczytów" (dirty reads), lecz nie zapobiega wyścigom sieciowym. Zgodnie z modelem *check-then-act*, w izolacji *Read Committed*, każda równoległa transakcja analizuje zatwierdzony stan bazy sprzed momentu własnego rozpoczęcia. W rezultacie wszystkie 10 zapytań zrealizowanych asynchronicznie widziało wolną salę. Brak wymuszonej serializacji przez klauzulę SELECT ... FOR UPDATE doprowadził do asynchronicznego utworzenia duplikatów.

Fragment kodu (Naprawa błędu w aplikacji za pomocą blokady FOR UPDATE):

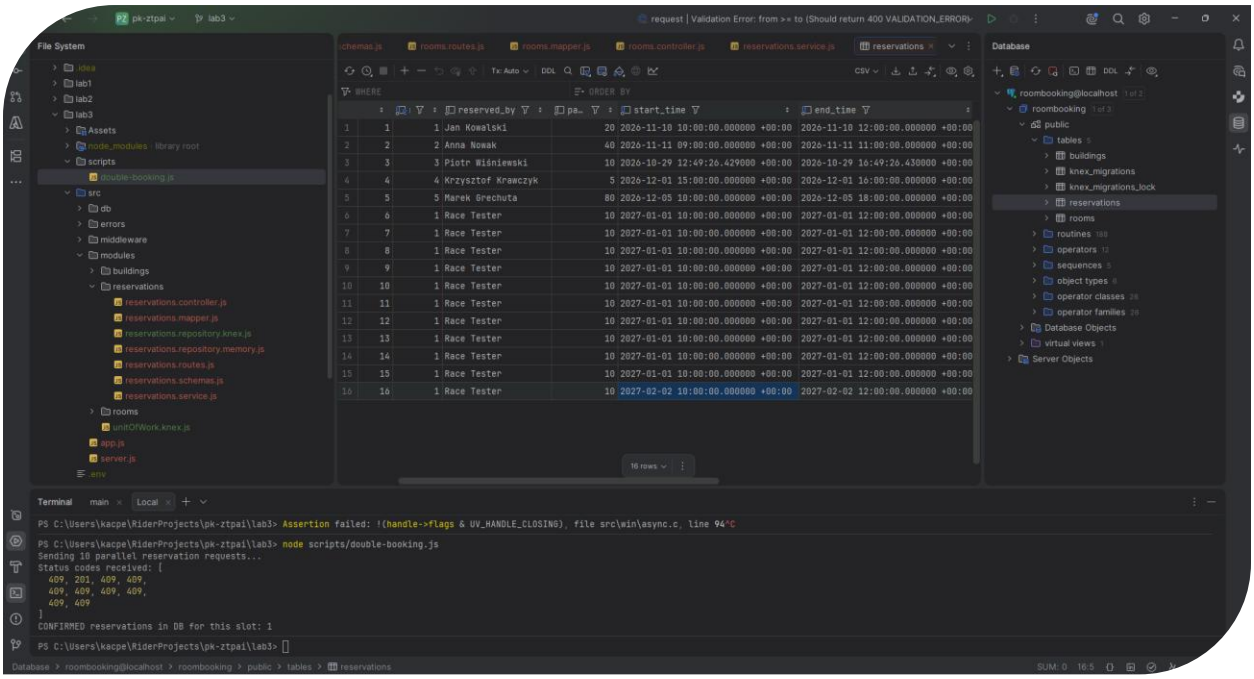
```
// BLOKADA APLIKACYJNA: Trzymamy zasób na wyłączność na czas transakcji (SELECT FOR UPDATE)
await rooms.lockById(input.roomId);

const { items: activeReservations } = await reservations.list({
    roomId: input.roomId,
    status: "CONFIRMED"
});

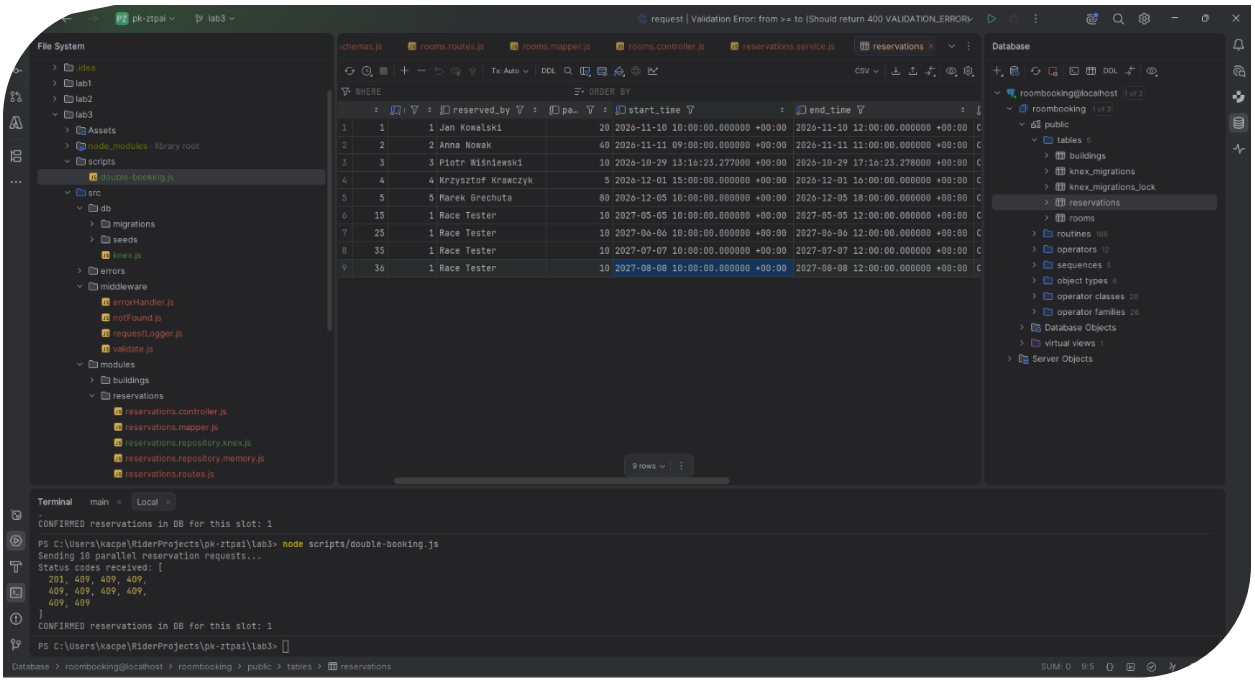
// Wątki ustawiły się w kolejce – blokada jest zwolniona dopiero PO zapisaniu rekordu
// Każdy kolejny pobiera DANE ZAKTUALIZOWANE i prawidłowo odbija się od ConflictError
```



Zrzut ekranu wyników skryptu (Część A) z wyłączonymi zabezpieczeniami - aplikacja odesłała kody 201 i wygenerowała w bazie duplikaty.



Zrzut ekranu wyników skryptu (Część B) z aktywnym SELECT FOR UPDATE na sali. Aplikacja poprawnie obsłużyła ruch tworząc tylko jedną rezerwację (jeden kod 201 i dziewięć poprawnie obsłużonych kodów 409).



Zrzut ekranu z odpiętą logiką serwera (brak weryfikacji aplikacji) zdanego wyłącznie na łaskę Exclude Constraints bazy danych. Błąd constraintu 23P01 silnika został poprawnie przechwycony i zmapowany na status 409 bez wywoływania ogólnego błędu serwera 500.